

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XI
Error Handlling



Disusun Oleh:
Alwisyah Putra 105224052

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

DAFTAR ISI

I. PENDAHULUAN.....	3
II. VARIABEL.....	3
1. Reservasi.java.....	3
2. Kereta.java.....	4
3. DataPenumpangTidakValidException.java.....	4
4. RuteTidakDitemukanException.java.....	4
5. TiketHabisException.java.....	5
6. App.java.....	5
III. CONSTRUCTOR & METHOD.....	6
1. Reservasi.java.....	6
2. Kereta.java.....	6
3. DataPenumpangTidakValidException.java.....	7
4. RuteTidakDitemukanException.java.....	7
5. TiketHabisException.java.....	7
IV. DOKUMENTASI DAN PEMBAHASAN CODE.....	8
1. Reservasi.java.....	8
2. Kereta.java.....	10
3. DataPenumpangTidakValidException.java.....	11
4. RuteTidakDitemukanException.java.....	11
5. TiketHabisException.java.....	12
6. App.java.....	13
V. KESIMPULAN.....	14
VI. DAFTAR PUSTAKA.....	15

I. PENDAHULUAN

Pada take home task kali ini, kami diminta untuk membuat sebuah sistem reservasi tiket kereta api yang menggunakan paradigma pemrograman berorientasi objek (OOP) sesuai dengan dunia nyata, di mana setiap entitas seperti kereta, penumpang, dan transaksi pemesanan dapat dimodelkan sebagai objek yang saling berkaitan. Tantangan dalam tugas ini adalah penempatan error handling yang tepat agar aplikasi tidak crash akibat input yang tidak valid dari user. Kami harus merancang beberapa exception custom, baik unchecked maupun checked, untuk menangani berbagai skenario kesalahan, seperti NIK yang tidak valid, kode kereta yang tidak ditemukan, hingga ketersediaan kursi yang tidak mencukupi. Selain itu, sistem juga harus mampu menangani error bawaan Java seperti InputMismatchException saat pengguna memasukkan data dengan tipe yang salah.

II. VARIABEL

1. Reservasi.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	daftarKereta	List<Kereta>	Menyimpan kumpulan objek kereta dalam bentuk list
2.	kodeKereta	String	Parameter untuk menerima input kode kereta dari pengguna
3.	nik	String	Parameter untuk menerima input NIK penumpang
4.	namaPenumpang	String	Parameter untuk menerima input nama penumpang
5.	jumlahTiket	int	Parameter untuk menerima input jumlah tiket yang dipesan
6.	i	int	Variabel counter pada loop for untuk iterasi karakter NIK
7.	keretaDitemukan	Kereta	Menyimpan objek kereta yang cocok dengan kode input, nilai awal null
8.	k	Kereta	Variabel iterasi pada enhanced for loop saat mencari kereta dalam daftarKereta

2. Kereta.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	kodeKereta	String	Menyimpan kode unik kereta (contoh: "K01", "K02")
2.	namaKereta	String	Menyimpan nama kereta (contoh: "Argo Bromo", "Parahyangan")
3.	rutePerjalanan	String	Menyimpan rute perjalanan kereta (contoh: "JKT - SBY", "JKT - BDG")
4.	sisaKursi	int	Menyimpan jumlah kursi yang masih tersedia untuk dipesan
5.	kodeKereta (parameter)	String	Parameter constructor untuk menerima input kode kereta saat pembuatan objek
6.	namaKereta (parameter)	String	Parameter constructor untuk menerima input nama kereta
7.	rutePerjalanan (parameter)	String	Parameter constructor untuk menerima input rute perjalanan
8.	sisaKursi (parameter)	int	Parameter constructor untuk menerima input jumlah sisa kursi
9.	jumlah	int	Parameter method kurangiKursi() untuk menerima jumlah tiket yang akan mengurangi sisa kursi

3. DataPenumpangTidakValidException.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	message	String	Parameter constructor untuk menerima pesan error yang akan ditampilkan saat exception dilempar

4. RuteTidakDitemukanException.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	message	String	Parameter constructor untuk menerima pesan error yang akan ditampilkan saat exception dilempar

5. TiketHabisException.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	namaKereta	String	Menyimpan nama kereta yang tiketnya habis, digunakan untuk memberi info spesifik saat error
2.	sisaKursi	int	Menyimpan jumlah sisa kursi yang tersedia, digunakan untuk memberi info spesifik saat error
3.	message	String	Parameter constructor untuk menerima pesan error utama
4.	namaKereta (parameter)	String	Parameter constructor untuk menerima nama kereta dari luar saat pembuatan exception
5.	sisaKursi (parameter)	int	Parameter constructor untuk menerima sisa kursi dari luar saat pembuatan exception

6. App.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	args	String[]	Parameter standar method main untuk menerima argumen dari command line
2.	scanner	Scanner	Objek untuk membaca input dari pengguna melalui keyboard
3.	reservasi	Reservasi	Objek class Reservasi yang menjadi pusat kontrol pemesanan tiket
4.	running	boolean	untuk mengontrol loop program, bernilai true program berjalan dan false saat keluar
5.	pilihan	int	Menyimpan input menu yang dipilih pengguna (1, 2, atau 3)
6.	k	Kereta	Variabel iterasi pada enhanced for loop saat menampilkan daftar jadwal kereta
7.	kode	String	Menyimpan input kode kereta dari pengguna saat memesan tiket
8.	nik	String	Menyimpan input NIK penumpang dari pengguna saat memesan tiket
9.	nama	String	Menyimpan input nama penumpang dari pengguna saat memesan tiket
10.	jumlah	int	Menyimpan input jumlah tiket yang dipesan pengguna

11.	e	InputMismatchException	Objek exception untuk menangkap error saat pengguna memasukkan
12.	e	DataPenumpangTidakValidException	Objek exception untuk menangkap error validasi data penumpang
13.	e	RuteTidakDitemukanException	Objek exception untuk menangkap error saat kode kereta tidak ditemukan
14.	e	TiketHabisException	Objek exception untuk menangkap error saat tiket tidak mencukupi

III. CONSTRUCTOR & METHOD

1. Reservasi.java

No.	Nama Method	Jenis	Fungsi
1.	Reservasi	Constructor	Menginisialisasi daftar kereta dengan data awal (Argo Bromo dan Parahyangan)
2.	getDaftarKereta	Instance	Mengembalikan list seluruh objek kereta yang tersedia
3.	pesanTiket	Instance	Memesan tiket dengan validasi NIK, pencarian kereta

2. Kereta.java

No.	Nama Method	Jenis	Fungsi
1.	Kereta	Constructor	Menginisialisasi objek kereta dengan kode, nama, rute, dan sisa kursi
2.	getKodeKereta	Instance	Mengembalikan nilai kode kereta
3.	getNamaKereta	Instance	Mengembalikan nilai nama kereta
4.	getRutePerjalanan	Instance	Mengembalikan nilai rute perjalanan kereta
5.	getSisaKursi	Instance	Mengembalikan nilai sisa kursi yang tersedia
6.	kurangiKursi	Instance	Mengurangi jumlah sisa kursi setelah pemesanan tiket

3. DataPenumpangTidakValidException.java

No.	Nama Method	Jenis	Fungsi
1.	DataPenumpangTidakValidException	Constructor	Menginisialisasi exception dengan pesan error untuk data penumpang tidak valid

4. RuteTidakDitemukanException.java

No.	Nama Method	Jenis	Fungsi
1.	RuteTidakDitemukanException	Constructor	Menginisialisasi exception dengan pesan error untuk kode kereta tidak ditemukan

5. TiketHabisException.java

No.	Nama Method	Jenis	Fungsi
1.	TiketHabisException	Constructor	Menginisialisasi exception dengan pesan error, nama kereta, dan sisa kursi
2.	getNamaKereta	Instance	Mengembalikan nama kereta yang tiketnya habis
3.	getSisaKursi	Instance	Mengembalikan jumlah sisa kursi yang tersedia
4.	getMessage	Instance	Override method dari parent class untuk mengembalikan pesan error lengkap dengan info kereta dan sisa kursi

6. App.java

No.	Nama Method	Jenis	Fungsi
1.	main	Static	Method utama sebagai entry point program, menjalankan menu CLI dan menangani seluruh exception

IV. DOKUMENTASI DAN PEMBAHASAN CODE

1. Reservasi.java

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  class Reservasi {
5      private List<Kereta> daftarKereta;
6
7      public Reservasi() {
8          daftarKereta = new ArrayList<>();
9          daftarKereta.add(new Kereta("K01", "Argo Bromo", "JKT - SBY", 50));
10         daftarKereta.add(new Kereta("K02", "Parahyangan", "JKT - BDG", 15));
11     }
12
13     public List<Kereta> getDaftarKereta() {
14         return daftarKereta;
15     }
16
17     public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket)
18         throws RuteTidakDitemukanException, TiketHabisException {
19
20         if (nik.length() != 16) {
21             throw new DataPenumpangTidakValidException("NIK harus 16 karakter");
22         }
23
24         for (int i = 0; i < nik.length(); i++) {
25             if (!Character.isDigit(nik.charAt(i))) {
26                 throw new DataPenumpangTidakValidException("NIK hanya boleh angka");
27             }
28         }
29
30         Kereta keretaDitemukan = null;
31         for (Kereta k : daftarKereta) {
32             if (k.getKodeKereta().equalsIgnoreCase(kodeKereta)) {
33                 keretaDitemukan = k;
34                 break;
35             }
36         }
37
38         if (keretaDitemukan == null) {
39             throw new RuteTidakDitemukanException("Kode kereta tidak ditemukan");
40         }
41
42         if (jumlahTiket > keretaDitemukan.getSisaKursi()) {
43             throw new TiketHabisException("Tiket tidak cukup",
44                 keretaDitemukan.getNamaKereta(), keretaDitemukan.getSisaKursi());
45         }
46
47         keretaDitemukan.kurangiKursi(jumlahTiket);
48         System.out.println("=== PEMESANAN BERHASIL ===");
49         System.out.println("Nama: " + namaPenumpang);
50         System.out.println("NIK: " + nik);
51         System.out.println("Kereta: " + keretaDitemukan.getNamaKereta());
52         System.out.println("Rute: " + keretaDitemukan.getRutePerjalanan());
53         System.out.println("Jumlah Tiket: " + jumlahTiket);
54         System.out.println("=====");
55     }
56 }
57
```

Di class Reservasi, saya menggunakan 2 library, yaitu ArrayList dan List, untuk memasukkan data kereta yang ada ke dalamnya. Dimulai dari membuat atribut List dengan nama daftarKereta dengan tipe data Kereta. Kemudian dibuat

konstruktor tanpa parameter, lalu di dalam konstruktor tersebut saya membuat ArrayList dan saya menambahkan 2 data kereta sesuai soal. Kemudian saya membuat getter karena atribut List saya adalah private, jadi harus ada getter.

Selanjutnya, saya membuat method pesanTiket dengan parameter String kodeKereta, String nik, String namaPenumpang, dan int jumlahTiket, dan saya mendefinisikan 2 exception untuk rute tidak ditemukan dan tiket habis. Pada inti method, program dimulai dengan kondisi: jika panjang NIK yang dimasukkan user bukan 16 karakter, maka akan melempar exception dengan parameter pesan "NIK harus 16 karakter". Lalu masuk ke for untuk memastikan setiap karakter yang diisi adalah angka; jika tidak, maka masuk ke if di mana program akan melempar exception dan muncul peringatan "NIK hanya boleh angka".

Kemudian dibuat variabel keretaDitemukan yang diset ke null, dan masuk ke for-each di mana akan mencari ke dalam kereta. Jika ketemu, maka keretaDitemukan diisi nilai k yang tadi ketemu di list, lalu program akan dihentikan. Jika kereta tidak ditemukan, maka akan melempar exception dengan pesan "Kode kereta tidak ditemukan". Jika jumlah tiket lebih besar dari sisaKursi, maka akan dilempar exception TiketHabis yang berisi pesan "Tiket tidak cukup". Kemudian jika mencukupi, maka sisaKursi akan dikurangi dengan jumlah tiket dan menampilkan struk.

2. Kereta.java

```
1  public class Kereta {
2      private String kodeKereta;
3      private String namaKereta;
4      private String rutePerjalanan;
5      private int sisaKursi;
6
7      public Kereta(String kodeKereta, String namaKereta, String rutePerjalanan, int sisaKursi) {
8          this.kodeKereta = kodeKereta;
9          this.namaKereta = namaKereta;
10         this.rutePerjalanan = rutePerjalanan;
11         this.sisaKursi = sisaKursi;
12     }
13
14     public String getKodeKereta() {
15         return kodeKereta;
16     }
17
18     public String getNamaKereta() {
19         return namaKereta;
20     }
21
22     public String getRutePerjalanan() {
23         return rutePerjalanan;
24     }
25
26     public int getSisaKursi() {
27         return sisaKursi;
28     }
29
30     public void kurangiKursi(int jumlah) {
31         this.sisaKursi -= jumlah;
32     }
33
34
35
36 }
```

Pada class Kereta, terdapat 4 atribut, yaitu kodeKereta, namaKereta, rutePerjalanan, dan sisaKursi, yang saya set ke private untuk menjaga keamanan data. Kemudian ada konstruktor yang berparameterkan 4 atribut awal untuk menginisiasi nilai. Setelah itu, terdapat 4 getter, di mana untuk setiap 1 atribut yang berstatus private memiliki 1 getter yang bertujuan untuk mengakses nilai dari atribut tersebut ketika kita menggunakan status atribut private. Kemudian, ada method kurangiKursi dengan parameter jumlah, di mana sisaKursi yang ada akan dikurangi dengan jumlah yang dimasukkan.

3. DataPenumpangTidakValidException.java

```
1 class DataPenumpangTidakValidException extends RuntimeException {  
2     public DataPenumpangTidakValidException(String message) {  
3         super(message);  
4     }  
5 }  
6  
7
```

Di sini saya membuat unchecked exception untuk data penumpang yang tidak valid, di mana ada konstruktor yang akan menangkap message dan menyimpannya ke induk class.

4. RuteTidakDitemukanException.java

```
1 class RuteTidakDitemukanException extends Exception {  
2     public RuteTidakDitemukanException(String message) {  
3         super(message);  
4     }  
5 }  
6
```

Di sini saya membuat checked exception untuk rute yang tidak ditemukan dengan mengextend Exception, di mana terdapat konstruktor dengan parameter message yang akan meneruskan ke induk dan akan disimpan.

5. TiketHabisException.java

```
1  class TiketHabisException extends Exception {
2      private String namaKereta;
3      private int sisaKursi;
4
5      public TiketHabisException(String message, String namaKereta, int sisaKursi) {
6          super(message);
7          this.namaKereta = namaKereta;
8          this.sisaKursi = sisaKursi;
9      }
10
11     public String getNamaKereta() {
12         return namaKereta;
13     }
14
15     public int getSisaKursi() {
16         return sisaKursi;
17     }
18
19     @Override
20     public String getMessage() {
21         return super.getMessage() + " | Kereta: " + namaKereta + " | Sisa Kursi: " + sisaKursi;
22     }
23 }
```

Di sini saya membuat custom error checked exception yang memiliki atribut namaKereta dan sisaKursi, di mana memiliki konstruktor yang berparameterkan message, namaKereta, dan sisaKursi untuk menginisiasikan nilai dengan parameter tersebut. Kemudian, karena seluruh atribut statusnya private, diperlukan 2 getter untuk mengambil nilainya, dan dibuat method override getMessage yang akan memunculkan pesan "| Kereta : " + namaKereta + " | Sisa Kursi : " + sisaKursi.

6. App.java

```
1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3
4 public class App {
5     public static void main(String[] args) throws Exception {
6         Scanner scanner = new Scanner(System.in);
7         Reservasi reservasi = new Reservasi();
8         boolean running = true;
9
10        while (running) {
11            System.out.println("=== JAVA EXPRESS ===");
12            System.out.println("1. Lihat Jadwal");
13            System.out.println("2. Pesan Tiket");
14            System.out.println("3. Keluar");
15            System.out.print("Pilih menu: ");
16
17            try {
18                int pilihan = scanner.nextInt();
19                scanner.nextLine();
20
21                switch (pilihan) {
22                    case 1:
23                        System.out.println("--- JADWAL KERETA ---");
24                        for (Kereta k : reservasi.getDaftarKereta()) {
25                            System.out.println(k.getKodeKereta() + " | " + k.getNamaKereta() + " | " + k.getRutePerjalanan() + " | Sisa: " + k.getSisaKursi());
26                        }
27                        System.out.println();
28                        break;
29
30                    case 2:
31                        System.out.print("Kode Kereta: ");
32                        String kode = scanner.nextLine();
33                        System.out.print("NIK: ");
34                        String nik = scanner.nextLine();
35                        System.out.print("Nama Penumpang: ");
36                        String nama = scanner.nextLine();
37                        System.out.print("Jumlah Tiket: ");
38                        int jumlah = scanner.nextInt();
39                        scanner.nextLine();
40
41                        reservasi.pesanTiket(kode, nik, nama, jumlah);
42                        break;
43
44                    case 3:
45                        running = false;
46                        break;
47
48                    default:
49                        System.out.println("Menu tidak tersedia");
50                }
51
52            } catch (InputMismatchException e) {
53                System.out.println("Input harus berupa angka");
54                scanner.nextLine();
55            } catch (DataPenumpangTidakValidException e) {
56                System.out.println("Error: " + e.getMessage());
57            } catch (RuteTidakDitemukanException e) {
58                System.out.println("Error: " + e.getMessage());
59            } catch (TiketHabisException e) {
60                System.out.println("Error: " + e.getMessage());
61                System.out.println("Kereta: " + e.getNamaKereta());
62                System.out.println("Sisa Kursi: " + e.getSisaKursi());
63            }
64        }
65
66        System.out.println("Terima kasih telah menggunakan JAVA EXPRESS");
67        scanner.close();
68    }
69 }
70
```

Di main, saya menggunakan 2 library, yaitu 'Scanner' untuk membaca input user dan 'InputMismatchException' untuk mengatasi input user yang tidak valid pada saat program dijalankan. Program dimulai dengan membuka Scanner, disusul dengan membuat objek reservasi, dan membuat variabel baru bernama running dengan tipe data boolean yang diset ke true untuk menampung status apakah nanti user ingin lanjut transaksi atau ingin keluar. Setelah itu, saya menggunakan do-while untuk membuat kondisi di mana selama running status true, maka dia akan meloop program yang ada di dalamnya. Baris selanjutnya di dalam loop akan ada 3 pilihan yang bisa user pilih, lalu program akan masuk ke blok try yang akan menjalankan program di dalamnya dimulai dari meminta input

user. Setelah user memasukkan pilihannya, maka program akan masuk ke percabangan case. Jika input user = 1, maka program akan memunculkan list jadwal dari kereta yang tersedia dengan sisa kursinya. Jika pilihan user = 2, maka user akan disuruh memasukkan (kode kereta, NIK, nama penumpang, dan jumlah tiket), kemudian program akan memanggil method pesanTiket pada class reservasi. Jika pilihan user = 3, maka status variabel running akan diset ke false sehingga program akan dihentikan, yang ditandai dengan munculnya ucapan terima kasih dan Scanner ditutup. Apabila user memasukkan pilihan yang tidak ada, maka program akan masuk ke cabang default, di mana akan menampilkan pesan error “Pilihan tidak valid”, dan program akan loop ke tampilan awal.

Jika pada saat input user melakukan kesalahan, seperti input yang tidak valid, maka program akan melempar ke blok catch InputMismatchException yang sesuai dengan errornya, lalu akan memunculkan pesan error “Input harus berupa angka”, dan blok catch akan memberi kolom input user lagi untuk diinputkan ulang. Jika input NIK tidak sampai 16 karakter, maka program akan melempar ke blok catch DataPenumpangTidakValidException. Jika nomor kereta yang diinput user tidak valid, maka program akan melempar ke blok catch RuteTidakDitemukanException. Dan jika tiket habis namun user masih keukeuh mesen, maka akan dilempar ke blok catch TiketHabisException.

V. KESIMPULAN

Berdasarkan laporan praktikum yang telah dibuat, dapat disimpulkan bahwa sistem reservasi tiket kereta "Java Express" ini berhasil menerapkan konsep error handling secara menyeluruh. Saya menggunakan kombinasi unchecked exception (seperti DataPenumpangTidakValidException) dan checked exception (seperti RuteTidakDitemukanException serta TiketHabisException) untuk menangani berbagai skenario kesalahan. Di dalam class App.java, saya menggunakan do-while loop dan blok try-catch untuk menangkap InputMismatchException, DataPenumpangTidakValidException, RuteTidakDitemukanException, dan TiketHabisException. Semua atribut pada class Kereta dan Reservasi saya set ke private dengan menyediakan getter masing-masing untuk menjaga keamanan data. Secara keseluruhan, program ini mampu berjalan tanpa crash meskipun user

melakukan input yang tidak valid, karena setiap kemungkinan error sudah ditangani dengan exception yang sesuai.

VI. DAFTAR PUSTAKA

Universitas Pertamina. (2026) Modul Praktikum Pemrograman Berorientasi Objek: Error Handling. Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.

Universitas Pertamina. (2026) Latihan Soal Error Handling Java OOP. Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.

Universitas Pertamina. (2026) Take Home Task: Sistem Reservasi Tiket Kereta Api "Java Express". Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.